

quantlab

Architecture, trade-offs and engineering practice

For programmers who know Python but have never seen this repository. No background in quantitative finance is needed, and nothing about the repository is assumed. This document explains why the code is organised the way it is; quantitative methods appear only where they explain a design choice.

Repository: github.com/ZhaorongDai/quantlab

Describes the main branch as of 2026-10-07

Contents

0	Before you start: the domain in five minutes
1	Overview: one pipeline, eighteen ordered layers
2	The one data contract: the panel
3	The component model: configs build objects, objects rebuild from configs
4	Reading is a stateless query
5	Extension points: template methods, hooks and wrappers
6	Base class or protocol
7	Invariants enforced by interface shape: "only t is visible"
8	Robust long-running jobs: resumable work and atomic writes
9	The run layer: a reproducible on-disk format
10	Isolating external dependencies
11	Tests that guard the architecture
12	Design records and project conventions
13	Trade-offs at a glance
14	Strengths, limitations and reserved directions
App.	A. Reading path B. Glossary C. A minimal end-to-end example

0 Before you start: the domain in five minutes

quantlab is a **Python backend library for quantitative equity research**: it looks for patterns in historical market data, trains models on them, and then simulates "what would have happened had we traded on the model". You can read this document without knowing finance; these few words are enough:

Domain term	What it means to a programmer
Bar	One record per stock per point in time (for example per day): open, high, low, close and volume. One bar is one time step.
Factor	A feature computed from market data, such as "how much the price rose over the last 20 days".
Label	The target a model predicts, such as "the return over the next 5 days". The y of machine learning.
Return model	A model (XGBoost, a neural network, ...) that predicts labels from factors, giving one score per stock per day.
Portfolio construction	Turning scores into "what fraction of capital each stock should take": the target weights.
Backtest	Simulating trading on those weights over historical data, to see how the account value would have moved.

Two problems of this domain bear directly on software design. Understand them and you understand half of this repository:

Problem one: look-ahead bias, or "peeking at the future"

When simulating the decision of day t , if the code reads any data from after t (even just "this stock will be halted tomorrow"), the backtest looks better than it should, and **nothing raises an error**. It is a silent class of bug that code review rarely catches completely. Much of this repository is designed so that such bugs cannot be written in the first place (chapter 7).

Problem two: reproducibility

A research result depends on data, parameters and code together. Data is appended to and revised, parameters are scattered across objects, and code changes every day. Three months later, "how was this curve produced?" often has no answer. This repository makes reproducibility a layer of its own (chapters 3 and 9).

~83k

lines of library code
(quantlab/)

~100k

lines of tests, about 3,000
test functions

18

ordered layers

28

architecture decision
records

Stack. Python ≥ 3.13 with `uv` for dependencies; xarray + Zarr as the data format (Zarr is a chunked on-disk format for N-dimensional arrays); factors computed with KunQuant (a library that compiles factor formulas to native code) and Polars; models with PyTorch, XGBoost and pytabkit; portfolio optimisation with cvxpy;

trade simulation with vectorbt; optional experiment tracking with W&B or MLflow. There is no web framework and no server process: it is a backend library.

Where the data comes from. Several market-data vendors are supported, such as WRDS (a data platform widely used in academia, which serves the CRSP US stock history and the TAQ tick quotes) and Sharadar (commercial US equity data). A caller can also pass in their own pandas or polars DataFrame.

1 Overview: one pipeline, eighteen ordered layers

1.1 The pipeline

The project's core value fits in one sentence: **one connected pipeline, driven by configuration and reproducible, whose modules are composed through clear input and output contracts so that any stage can be replaced on its own.**

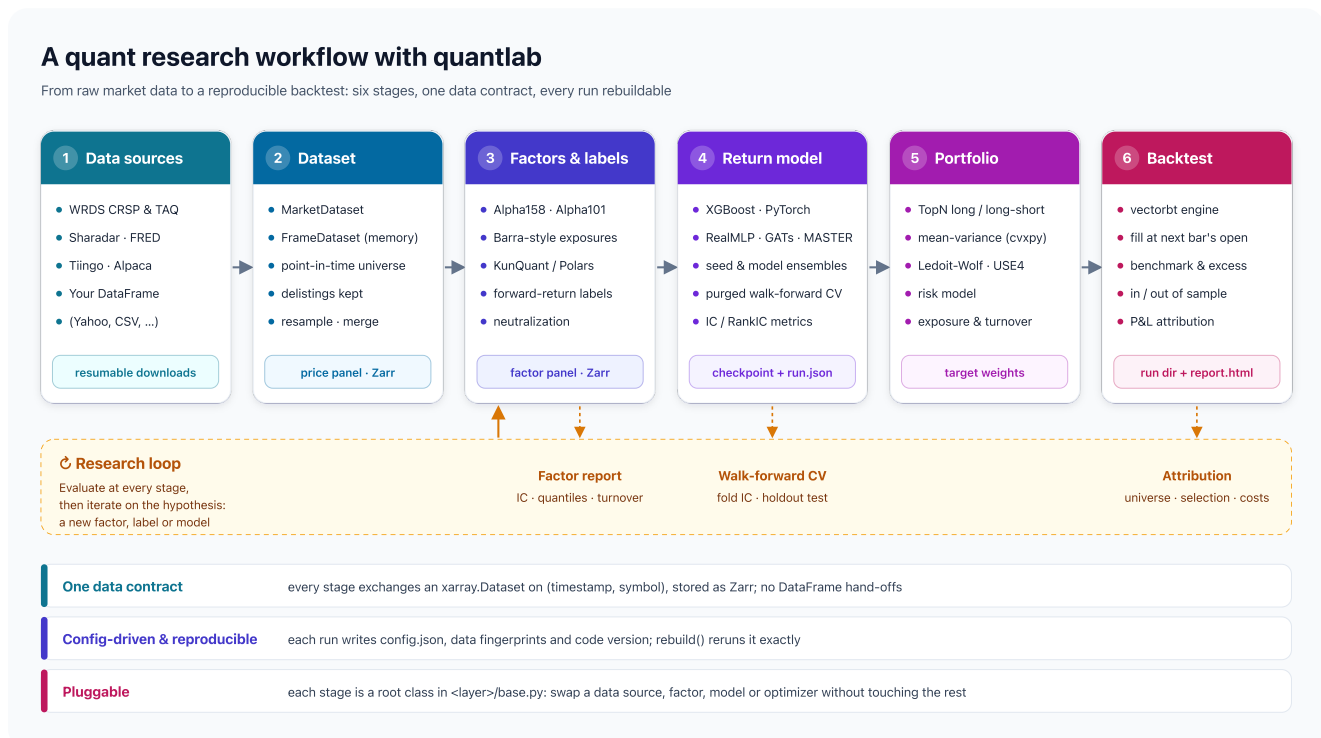


Figure 1-1. The research workflow. Each stage is the root class of one layer; the output of one stage is the input of the next.

Stage	Root class (in <layer>/base.py)	Input → output
Acquisition	Acquisition	vendor API → raw Parquet files
Dataset	BaseDataset / MarketDataset	raw files → price panel (stored as Zarr)
Factor / label	Factor / Forward	price panel → factor panel
Return model	BaseModel	factor panel + label panel → prediction panel
Risk model	FactorRiskModel	risk characteristics + past returns → a risk (covariance) forecast per bar
Portfolio construction	PortfolioConstructor	one bar's predictions + current holdings → target weights
Backtest	BaseBacktester	weights + prices → simulation, run directory

A "panel" is this project's name for data of one fixed shape; chapter 2 is about it.

1.2 The layer chain

The code is organised in layers: each sub-package of `quantlab/` is one layer. All layers form **one ordered chain**, and a layer may import only layers earlier in the chain. A deferred import inside a function counts too, and there are **no exceptions**.

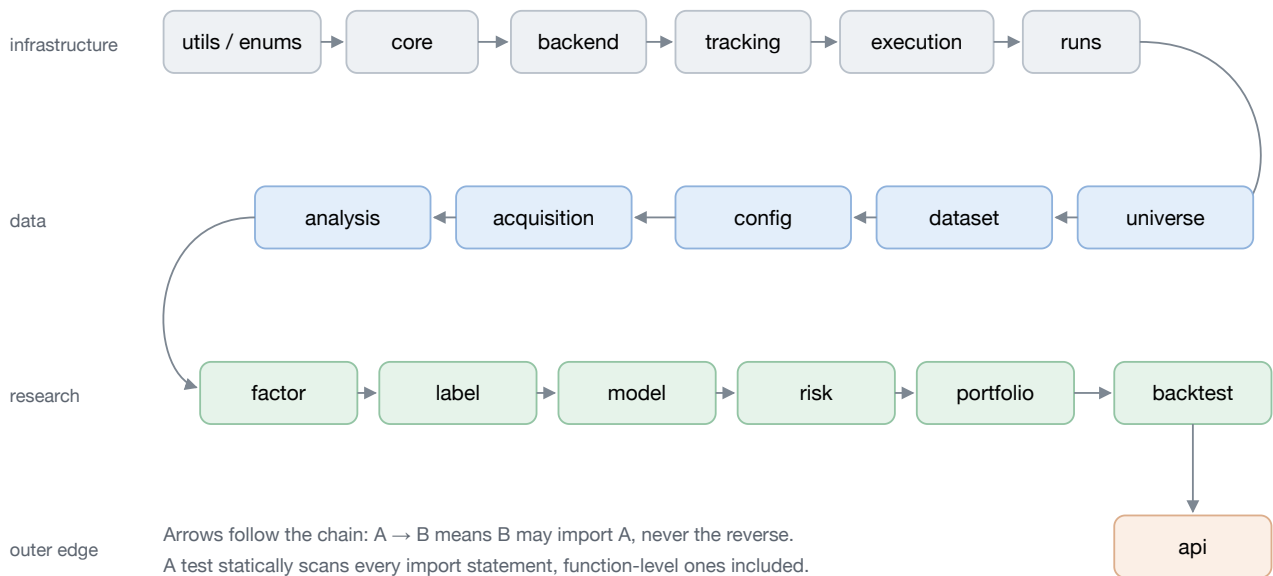


Figure 1-2. The order of the 18 layers. The last one, `api`, is a facade for users who only want to work with DataFrames.

Layer	Responsibility
utils / enums	Generic helpers (atomic writes, date ranges, timers, ...) and shared enumerations
core	The component rule: how an object is serialised to a config and rebuilt from it (chapter 3)
backend	Storage backends: reading and writing Zarr and Parquet
tracking	The experiment-tracking interface and its W&B and MLflow adapters
execution	The rules that simulate how the market fills orders
runs	Reading and writing run directories: how training and backtest results are stored (chapter 9)
universe / dataset	Stock universes; turning raw data into panels
config / acquisition	Data-root configuration; downloading from each vendor
analysis	Factor reports
factor / label / model	Factors, labels, return models
risk / portfolio / backtest	Risk models, portfolio construction, backtests
api	Convenience functions that take and return DataFrames

A few positions in the chain may be surprising; each has a reason:

- **runs comes early**, before dataset and model. Both the model layer and the backtest layer **write** run directories, so it must come before both; and since runs never imports either of them, opening a run directory never loads torch.
- **execution comes early too**. The backtest engine uses it to simulate fills, and portfolio construction uses it to work out what is actually held. Both must share **one** copy of these rules, so it sits before both.
- **risk sits between model and portfolio**. A risk model makes no investment decision, and risk analysis does not need the portfolio layer.

1.3 The layout inside a layer

Every layer follows the same layout; read one and you have read them all:

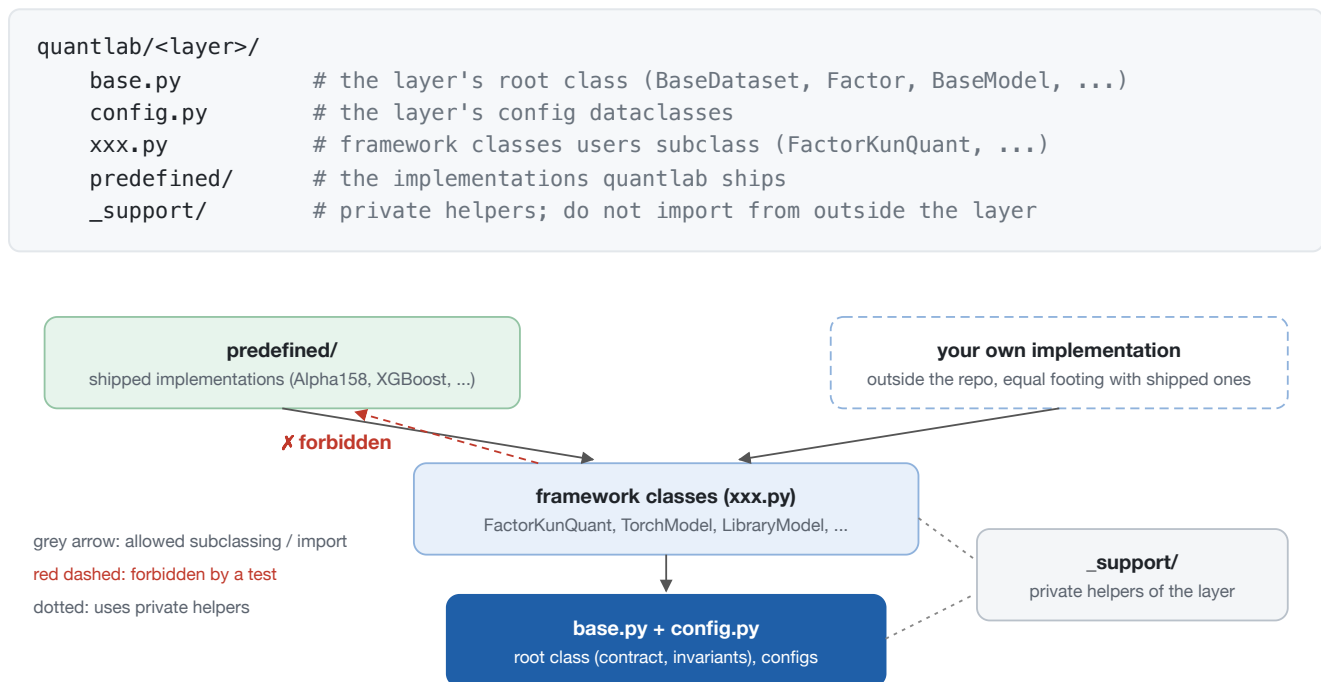


Figure 1-3. Inside one layer. Shipped and user implementations depend only on the framework; neither the framework nor the root class depends on any shipped implementation.

Convention	Meaning and reason
Framework modules never import <code>predefined/</code>	A user's own factor or model depends only on the framework, never on a shipped implementation. Shipped and user implementations stand on equal footing. Enforced by a test.
The top level of <code>dataset/</code> and <code>acquisition/</code> reads like a menu	Every top-level entry of these two directories is one complete, usable thing (a dataset, a vendor); helpers go into <code>_support/</code> . Browsing the directory is browsing the feature list.
No <code>__init__.py</code> re-exports	Every import spells the full path, e.g. <code>from quantlab.factor.predefined.alpha158 import Alpha158Stock</code> . Each name has one spelling, so <code>grep</code> always finds its definition.

A few packages' <code>__init__.py</code> is the entry module	For example <code>quantlab/dataset/crsp/__init__.py</code> is the CRSP dataset's main module, with its submodules beside it. The rule is "the <code>__init__</code> is the module itself", never "the <code>__init__</code> forwards other modules".
Some <code>__init__.py</code> files must be empty	A non-empty <code>__init__</code> runs on every import beneath it. One guarantee ("the credential-free source inspector never loads a vendor client, not even indirectly") depends on a few empty files, and a test checks them (chapter 10).
Root classes and configs load no torch	Importing any layer's <code>base.py</code> or <code>config.py</code> does not load torch. A test checks <code>'torch' in sys.modules</code> in a subprocess.

1.4 How one backtest flows through the code

A sequence diagram ties the layers together: after `backtester.run()` is called, each layer talks only to the contracts next to it.

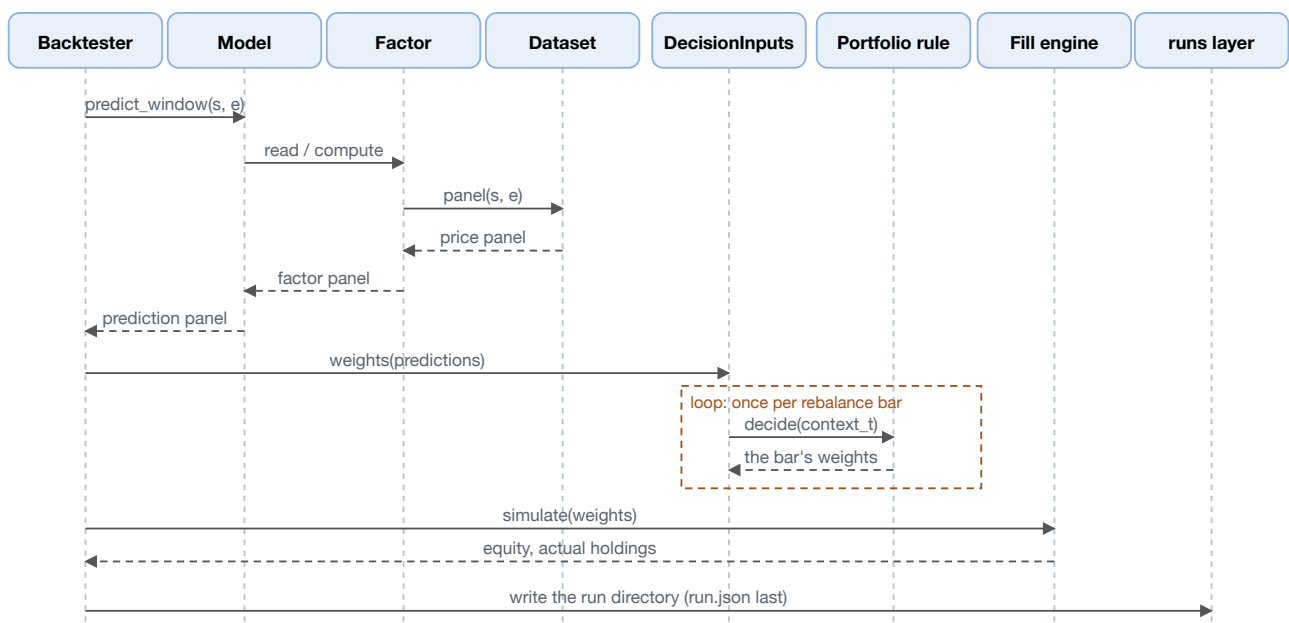


Figure 1-4. The order of calls between layers in one `run()` (simplified). The whole run sits inside a `DataRecorder`, which records every dataset and factor read (chapter 9).

2 The one data contract: the panel

2.1 What a panel is

Layers exchange exactly one kind of data: an `xarray.Dataset` with **exactly** two dimensions, (`timestamp`, `symbol`) (time × stock), and one variable per field. The project calls it a **panel**. On disk, every Zarr store (a "store" below) holds exactly one such panel.

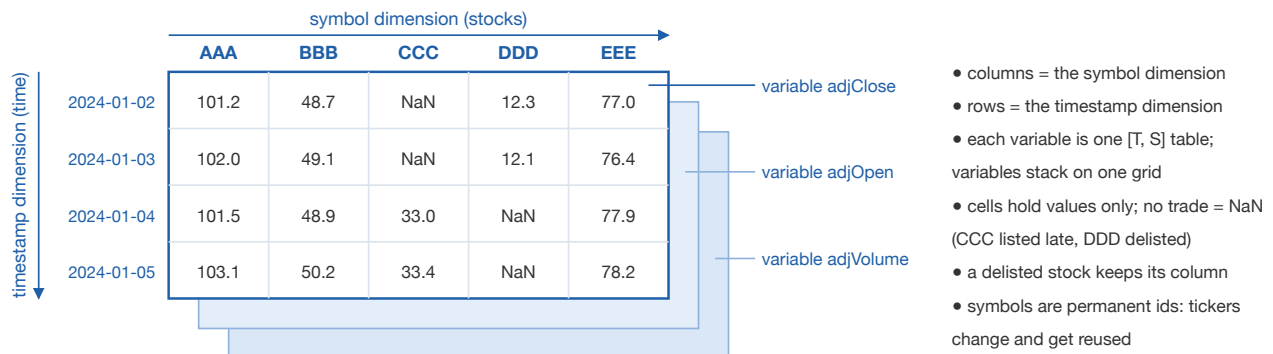


Figure 2-1. A panel: its two dimensions are timestamp (rows) and symbol (columns), the grid holds values, and several variables stack on one grid. The adj prefix marks prices adjusted for splits and dividends.

```
<xarray.Dataset>
Dimensions:    (timestamp: 400, symbol: 16)
Coordinates:
  * timestamp  (timestamp) datetime64[ns] 2022-01-03 ... 2023-07-14
  * symbol     (symbol) <U3 'S00' 'S01' ... 'S15'
Data variables:
  adjOpen     (timestamp, symbol) float64 49.74 50.21 ...
  adjClose    (timestamp, symbol) float64 50.13 49.87 ...
```

Why keep delisted stocks?

Studying only the companies that exist today leaves out those that went bankrupt or were delisted, and gives conclusions that are too optimistic (survivorship bias). A dense panel keeps a delisted stock as a column, so models and backtests "see" it.

2.2 Why xarray rather than DataFrames

- **The shape is the contract.** In a long DataFrame (one row per timestamp and symbol), "no data for this stock on this day" has two representations: a missing row, or a row of NaN. Mixing them is a rich source of bugs. A dense panel has one representation.
- **Cross-sectional operations vectorise naturally.** The most common operation in this domain is "for each day, rank or standardise across all stocks". On a `[T, S]` array that is one operation along the symbol axis; on a long table it is a groupby.
- **Models consume it directly.** A panel becomes a `[T, S, F]` torch tensor (F factors) with no conversion, and the deep models are designed around that shape.
- **Zarr stores in chunks and loads on demand.** A whole-market factor panel can be 3,270 days $\times$ 13,015 stocks $\times$ 169 factors, about 29 GB; xarray + Zarr reads only the part needed.

2.3 Who guards the shape

"Only one shape" holds only if code enforces it. Two places do:

1. **The storage backends' `get_xarray_dataset(indexes)`** : asked for `["timestamp", "symbol"]` , both the Zarr and the Parquet backend drop variables on other dimensions and transpose to the requested order; asking for a dimension the data lacks raises `ValueError` . A dedicated test covers both backends.
2. **Strict merge semantics:** when several panels are outer-joined on both dimensions, a cell holding a value in two inputs is an error, **never** resolved silently by input order.

2.4 The one exception: a facade for DataFrame users

Many users hold only pandas or polars DataFrames and want one capability (computing factors, say, or a backtest) without adopting the project's stores and configs. `quantlab.api` gives them functions that take a DataFrame and return one. Design points:

- Conversion between DataFrames and panels happens **only** in the api layer. The library's own result types stay pure xarray and do not know the api layer exists. "Layers exchange only panels" governs the inside of the pipeline; the api layer is its outer boundary.
- An incoming DataFrame becomes a `FrameDataset` : a real `MarketDataset` subclass whose panel lives in memory. Factors and backtests therefore run through their **public** interfaces; the api layer calls no private method, so internal refactoring does not break it.
- Nothing is written to disk by default; with `output_dir` , the inputs are copied into the result directory so the run can still be rebuilt later.
- A pandas DataFrame in gives a pandas DataFrame out, and likewise for polars; `as_xarray=True` returns xarray instead.

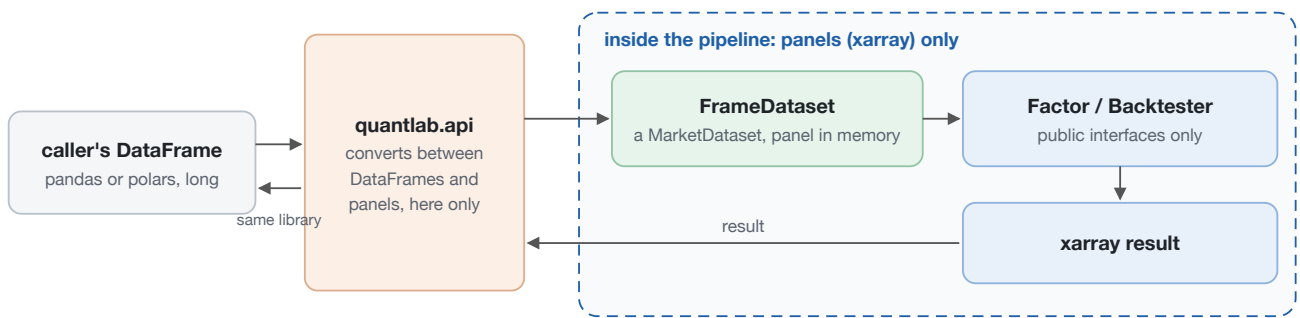


Figure 2-2. The api layer is the outer boundary of the pipeline: conversion happens at the boundary, and inside there are still only panels.

3 The component model: configs build objects, objects rebuild from configs

3.1 Every object is built from one config dataclass

Datasets, factors, labels, models, portfolio rules, backtesters, experiment trackers: almost every object in the project is a **component**. It is built from **one** config dataclass and keeps it as its `config` attribute.

Configs nest: a factor's config holds the dataset object it reads, a model's config holds its lists of factors and labels, a backtest's config holds the price dataset and the model.

Every parameter that changes results (a window length, a filter threshold, a fee) is a config field, never a constant in code, for a simple reason: a config can be saved, a constant cannot.

3.2 Declaring which fields hold components

The hard question when serialising nested objects is "how do we know a field holds an object that must be recursed into?". Guessing from key names, or treating any dict with a "name" key as an object, scatters the rule everywhere. This project has one answer: **declare it on the dataclass field**.

```
# quantlab/label/config.py (excerpt)
@dataclass(kw_only=True, frozen=True)
class ForwardConfig(FrozenConfig):
    factor: "Factor" = component()      # this field holds one component
    span: int                          # plain data
    delay: int = 1
    name: str | None = None             # the class's import path, filled in by the object

# quantlab/model/config.py (excerpt)
@dataclass(frozen=True)
class ModelConfig(FrozenConfig):
    factors: list["Factor"] = component(many=True)  # holds several components
    labels: list["Factor"] = component(many=True)
    tracker: Tracker = component(default_factory=NullTracker)
    hyperparameters: dict ...                # a plain dict, never recursed into
    ...
```

That one declaration drives three things:

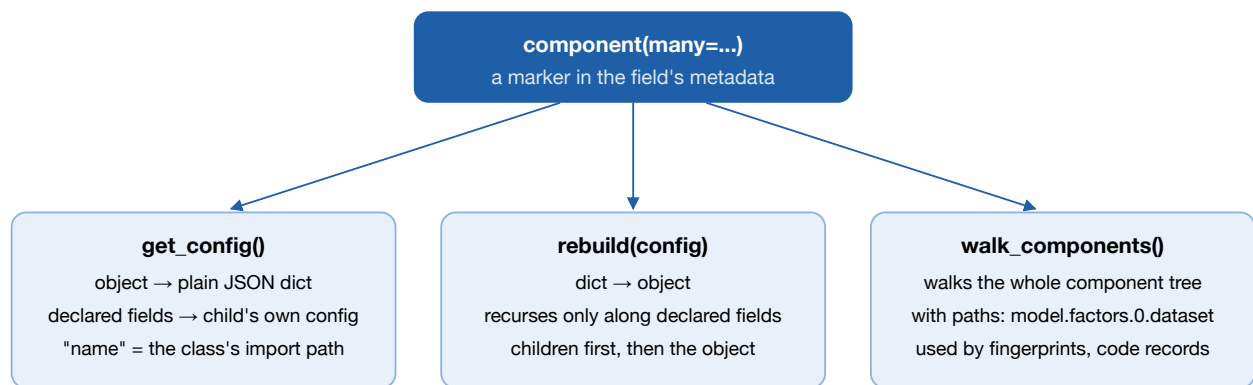


Figure 3-1. One declaration, three uses. Objects are recognised neither by key name nor by the shape of a dict.

```

label = Forward(ForwardConfig(factor=momentum, span=5))
saved = label.get_config()
saved["factor"]["name"] # 'quantlab.factor.predefined.momentum.Momentum'
saved["factor"]["dataset"]["name"] # 'quantlab.dataset.spot.SpotKlineDataset'
rebuild(saved) == label # True
  
```

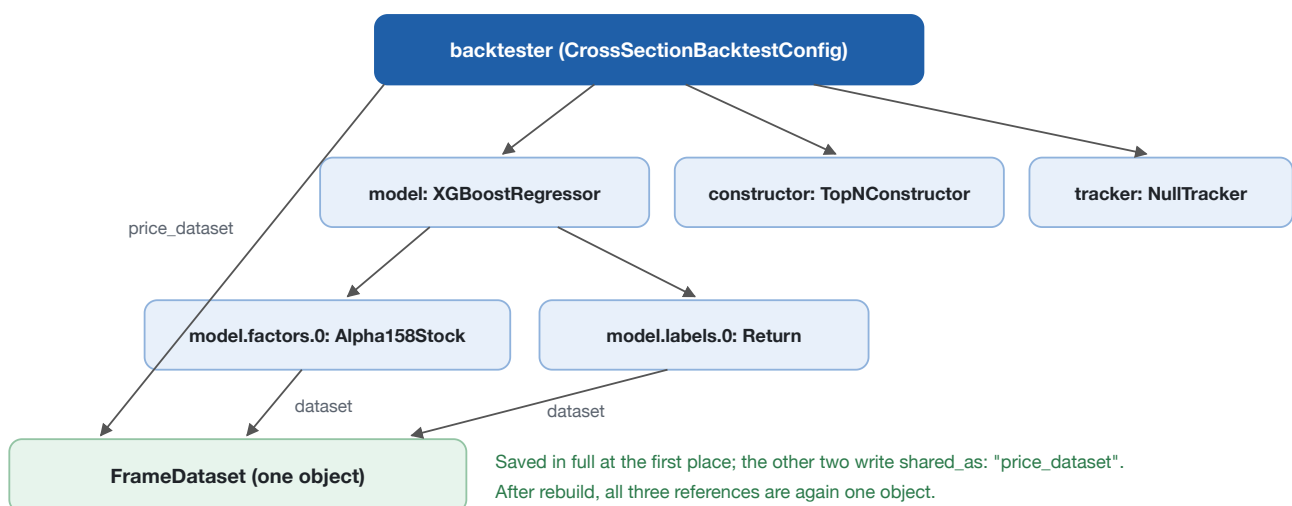


Figure 3-2. The component tree of a backtester. Node names are component paths; one dataset object is referenced from three places and stays shared through saving and rebuilding.

Details worth knowing

- **Safety:** only a class that declares a config class (`config_cls`) is ever rebuilt, so a tampered JSON file cannot use `rebuild` to call an arbitrary importable function.
- **Shared references survive:** when one dataset object is referenced by both a factor and the backtester, the second place is written with a `"shared_as"` key naming the first; one `rebuild` call turns them back into **one** object, so the rebuilt object graph matches the saved one.
- **Unknown fields are refused:** a key the config class does not have raises instead of being ignored.
- **Strict rebuild:** rebuilding a backtester requires every config field to be present instead of filling gaps from today's defaults. A default changed since the run would silently produce a different backtest.
- **The class path is the contract:** a class is found by its import path, so moving or renaming a class makes saved configs unloadable. The project accepts this cost (see 12.3, "No backward compatibility").

3.3 Configs are frozen

Every config class derives from `FrozenConfig` and is a `frozen=True` dataclass that cannot be changed after creation. An object "completes" the config it is given (it records its class name, resolves the list of factor names, ...) by **returning a completed copy**, never by changing the one the caller passed in. The next chapter explains why that matters.

4 Reading is a stateless query

The pipeline reads date ranges everywhere: a model reads 2016–2021 factors to train, a backtest reads 2023–2024 prices. The rule: **a date range is always an argument of the read method, and is never written into another object's config.**

4.1 The read interface

```
panel = dataset.panel(start, end, symbols=None, variables=None) # a dataset answers a query
values = factor.read(start, end) # read values already computed into the factor's store
values = factor.compute(start, end) # compute from the inputs
```

4.2 Why dates do not go into configs

A natural-looking alternative is to set `config.start_date / end_date` on the object and then call `read()`, with the factor passing the dates on to its dataset and the dataset caching data for the new range. In a data pipeline the trouble with this is **shared mutable state**:

- One dataset is often referenced by several factors and a backtester at once. Whoever wrote the dates last wins, and results depend on call order;
- if a dataset caches and narrows its data in place for a range, other users read the narrowed data, and can only defend themselves with forced re-reads or deep copies;
- one dataset object cannot serve two users with different ranges, and there is nowhere to merge several datasets into one input.

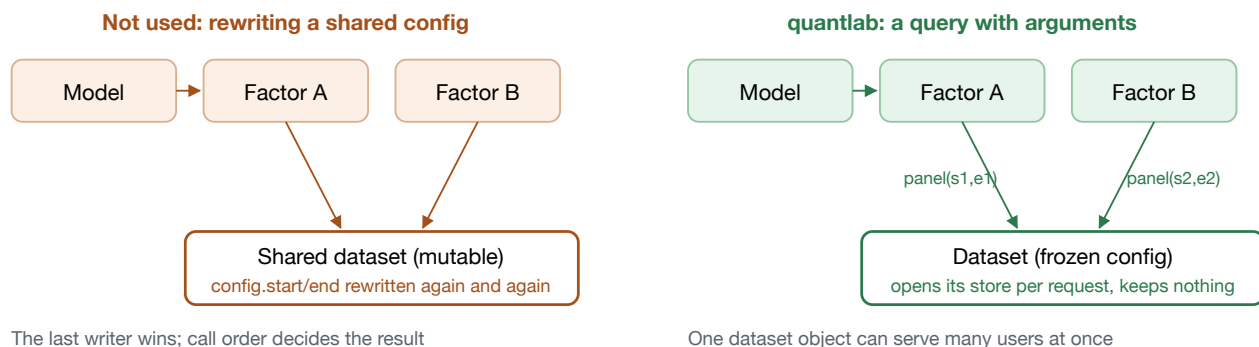


Figure 4-1. Two ways to pass dates. quantlab uses only the one on the right.

4.3 Companion rules

- **Reads keep no state:** a dataset opens its store on each request and keeps nothing afterwards. Only the build path (raw files to store) holds data in memory.
- **Factor configs have no date fields:** the range a factor store covers is an argument of **building** the store, and is recorded beside it.
- **Warm-up is counted in bars, not calendar days:** rolling computations read some history before the start date, its length given by the `warmup_bars` field. The dataset answers "which bar lies N bars before this date". Counting bars avoids "30 days" meaning different things on daily and minute data.
- **The outer object's dates win:** a model passes its dates to its factors with each request, and a backtester passes its window to the model.

Why are `read` and `compute` two methods, not "read if there is a store, compute otherwise"?

With one method choosing automatically, the caller could not tell whether a value was read or computed, and the data fingerprints of chapter 9 need to know exactly that.

A bonus

A factor can take **a list** of datasets and merge them into one input panel (renamed to shared variable names, then outer-joined; a conflict is an error). And "no shared mutable state" is a precondition for a future multi-user service.

5 Extension points: template methods, hooks and wrappers

One long-term goal is editing and testing factors and models online, so "how much code does a new factor or model take?" is a core design metric. The general answer: **the root class implements the whole lifecycle (the template method pattern), and a subclass fills in a few hook methods.**

5.1 Factors: two compute backends, one hook each

Intermediate class	A subclass implements only	Characteristics
FactorKunQuant	<code>_get_factor_func()</code> : return a KunQuant computation graph	A factor is a declarative graph of operators (Builder / Input / Output / operators), compiled to native code just in time and run on several threads. The same graph computes a whole history in batch or updates bar by bar in streaming mode , so a factor written for research can later run on live data. The main backend.
FactorPolars	<code>_get_factor_lazyframe(lf)</code> : return a lazy Polars query	Batch only. For formulas easier to express as table operations; a complement to KunQuant.

```
class Momentum(FactorPolars):
    """N-bar price momentum, Close_t / Close_{t-n} - 1."""
    def _get_factor_lazyframe(self, lf):
        n = self.config.kwargs.get("n", 20)
        return lf.select(
            "timestamp", "symbol",
            (pl.col("Close") / pl.col("Close").shift(n).over("symbol") -
             1).alias(f"momentum_{n}"),
        )
# everything else (reading by date, warm-up, storing, extending, resampling, recording
# fingerprints) is done by the root class Factor
```

(Illustrative; the actual implementation is `quantlab/factor/predefined/momentum.py`.)

The root class `Factor` provides one lifecycle: `compute(start, end)` computes; `build` writes a store; `extend(end)` appends new dates; `read(start, end)` reads the store; `resample(freq, how)` aggregates to coarser bars (minute bars to daily, say); `analyze()` writes a factor report.

A platform detail: SIMD alignment on macOS

On macOS, KunQuant needs the number of stocks to be a multiple of 8. `FactorKunQuant` pads the symbol axis with all-NaN placeholder stocks to a multiple of 8 and cuts them off afterwards, but only when `sys.platform == "darwin"`. On Linux, 4, 5, 7, 8, 12 and 16 stocks were measured to work, so nothing is padded there. Padding happens only on the platform that needs it, and the measurement is written down in the docs.

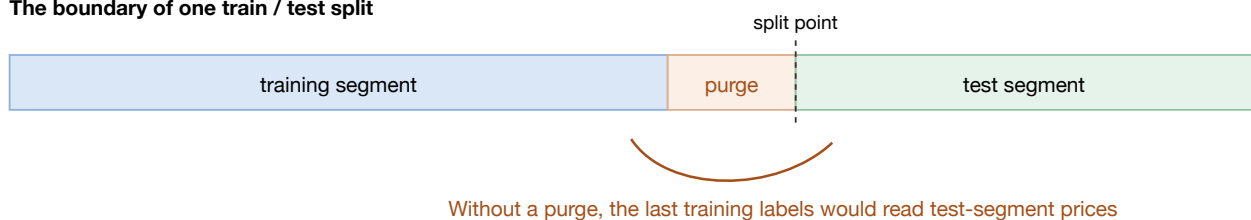
5.2 Labels: a wrapper, not a subclass

Mathematically a label is a factor moved forward in time: the return over the next 5 days is the value, 5 days later, of a "return over the last 5 days" factor. So labels get no class hierarchy of their own; a **wrapper** turns any factor into a label:

```
label = Forward(ForwardConfig(factor=any_factor, span=5, delay=1))
label.lookahead_bars() # 6 = delay + span: the label at t reads data up to 6 bars after t
```

`Forward` asks the wrapped factor for `lookahead` more bars, shifts the panel and trims it back to the request. It owns no store, so **one factor store serves as a feature directly and, wrapped, as a label**. A model accepts only `Forward` objects as labels and refuses them as features: whether something is a feature or a label depends on how it is used, not on its class, and a type check prevents mistakes.

The boundary of one train / test split



Purge length = the largest lookahead among the model's labels, **derived from the labels, not a parameter**

Applies at every split: train/validation, train/test, each walk-forward fold, the in/out-of-sample line of a backtest

Figure 5-1. The purge drops the last training bars whose labels would see the test segment. The lookahead is a queryable property of the label, and every split uses it.

Because "how many bars does a label look ahead" has one source, the label object, model splits, factor reports and the backtest's in/out-of-sample line all read it from there instead of each computing their own. The backtest engine also declares its fill delay, `fill_delay_bars`, and a backtest refuses to run when a label's `delay` differs from it.

5.3 Models: three levels of inheritance, as few hooks as possible

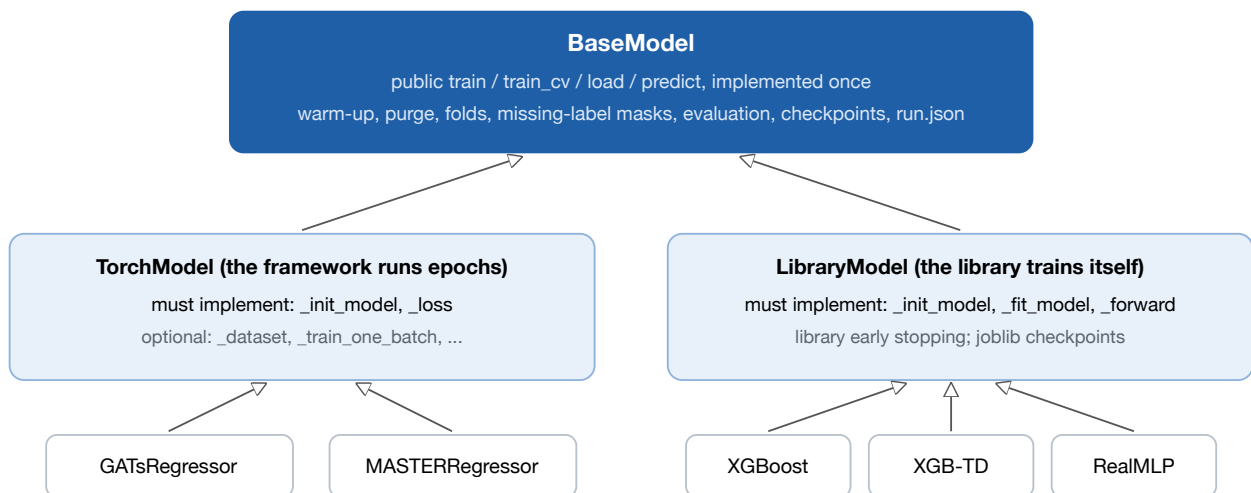


Figure 5-2. The model layer's inheritance. The framework owns the data contracts, a concrete model owns its learning strategy. The bottom row ships with quantlab.

The steps of one `train()` call; only "fit" belongs to the concrete model:

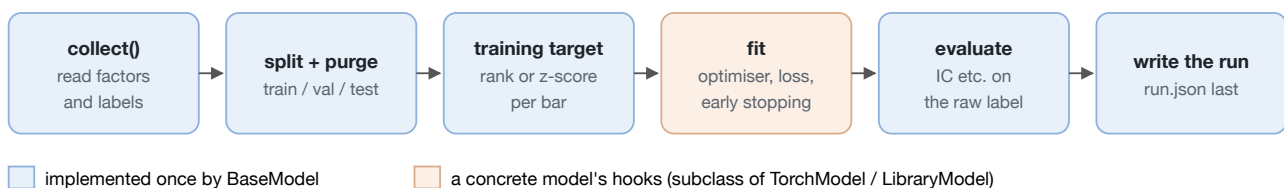


Figure 5-3. The steps of `train()`. Blue: implemented by the base class; orange: the concrete model's hooks.

The division of labour: **the base class owns the data contracts, the hooks own the learning strategy.**

Optimiser, loss and what one training step does are up to the concrete model; warm-up, purge, cross-validation folds, missing-label masks, metrics on the raw label, the checkpoint format and putting predictions back into `[T, S, L]` (time × stock × label) belong to the base class.

Torch models choose their sample shape

A torch model returns a standard `torch.utils.data.Dataset` from `_dataset()` to decide what one training step sees. The default is the whole cross-section of one bar (all stocks enter the network together, which suits models with attention between stocks); another option is one stock's time series, in the style of Qlib (Microsoft's open-source quantitative research platform). The key constraint: every sample carries `where = (t_idx, s_idx)`, so the base class puts predictions of any shape back into `[T, S, L]` with the same code, and **refuses** a set of predictions that misses or repeats a cell.

Deep models are not sized to a fixed set of stocks, because index membership changes every month: a network whose input is a fixed $S \times F$ could not predict a newly added stock. The network maps `[S_t, N, F]` (stocks at the bar × history length × factors) to `[S_t, L]`, and `S_t` may differ from bar to bar. A stock without a label stays in the cross-section as context and is left out of the loss.

A trade-off decided by measurement: training data in memory, or loaded from disk on demand?

Measured on the training server on a whole-market factor panel of about 29 GB: a cross-section model reads one bar in 15.4 ms from memory, 58 ms from a time-chunked cache (22 ms with 8 workers, still no faster than memory) and 99 ms from the raw stores; with the panel resident on the GPU, 0.1 ms. A sequence model reads an 800-sample batch in 2.7 ms from memory and 177 ms from the cache. Hence the rule: the panel goes on the GPU when it fits (in half the free GPU memory), otherwise in RAM, optionally as float16; **on-demand loading is considered only once a panel no longer fits in RAM.**

Known pitfalls written as tests

Evaluation loaders never drop the last partial batch; evaluation runs under `no_grad` and `eval()`; a one-stock cross-section works; training shuffles bars; rows with a missing label stay in the feature panel; the purge covers the label lookahead only, not the feature window. Each has a test.

5.4 Portfolio rules and covariance estimators

The portfolio-construction root class `PortfolioConstructor` asks for one method, `construct(context)`; the risk root class `CovarianceEstimator` asks for `estimate(...)` (the covariance of stock returns: the matrix describing how stocks' risks move together).

Both return **one** `InputDeclaration` value from `declared_inputs()`: how much history they read at a decision, which factors, which risk model. A rule that holds an estimator merges the two with `InputDeclaration.merged`. Describing every input need as one value means a new kind of input is a new field on that value, not a new method on two base classes forwarded layer by layer.

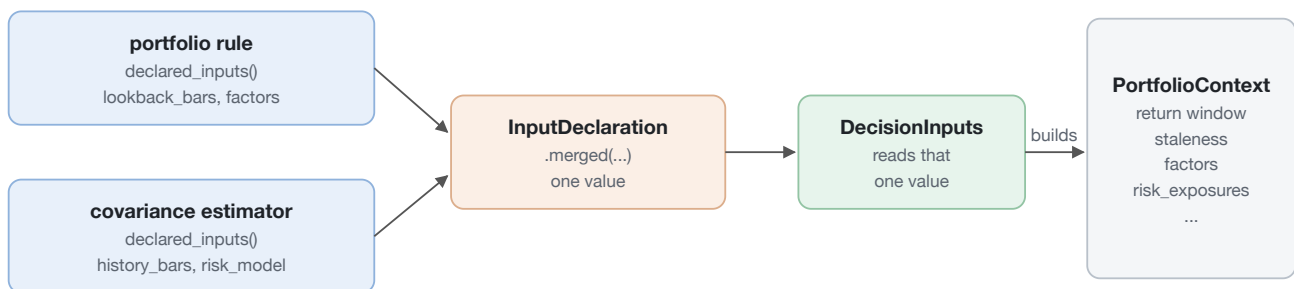


Figure 5-4. Merging input declarations: the rule and the estimator each declare what they need, and `DecisionInputs` reads one merged value to build each bar's context.

The mean-variance optimiser (a classic method that trades expected return off against risk) also opens two extension points: `reference_weights` (measure risk relative to a reference portfolio) and `risk_constraints` (constrain the risk term). The base optimiser leaves both unused; `ActiveRiskOptimizer` in `scripts/active_risk.py` is built on them, and a test checks that it works.

6 Base class or protocol

Python can express an interface in two ways: an abstract base class (by inheritance) or a `typing.Protocol` (structural typing: having the methods is enough). The project has an explicit rule for which to use:

The rule

Each layer's root class is the layer's contract, and implementations derive from it. **A Protocol is defined at a seam only when a second implementation appears there that meets the caller's needs but must not inherit the root class's code.** The Protocol is then defined beside its caller and lists only the members the caller uses.

6.1 The one instance: Predictor

A backtester needs a "signal source" for predictions. A single model (a `BaseModel` subclass) obviously qualifies, but so must an ensemble (such as `SeedEnsemble`: one config trained under several random seeds and averaged). An ensemble inheriting `BaseModel` would inherit a training loop, cross-validation logic and checkpoint rules it does not have, and would have to stub each one. So the backtest layer defines a `Predictor` protocol:

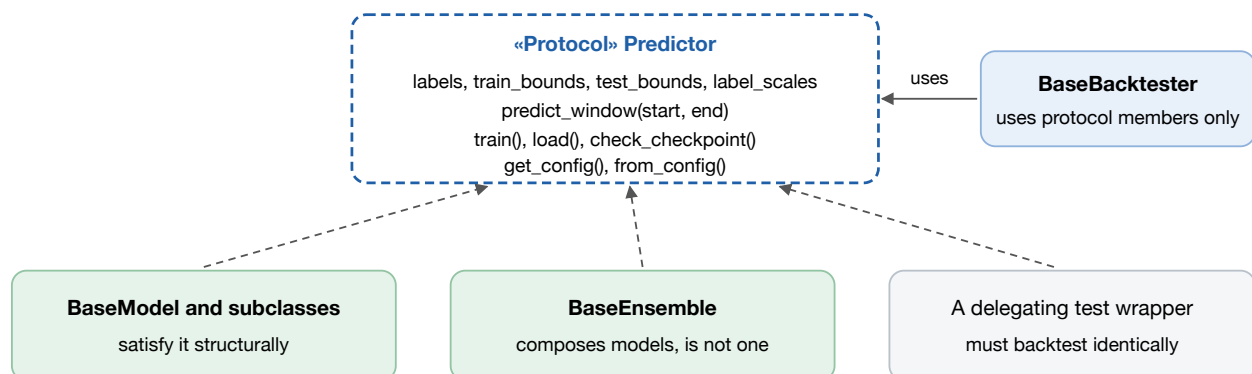


Figure 6-1. The Predictor protocol is defined beside its caller (the backtest layer); models and ensembles both satisfy it structurally.

- The protocol **deliberately exposes no `ModelConfig`**: an ensemble may combine quite different models (an XGBoost and a neural network, each with its own factors), which have no single config to expose.
- The project runs no static type checker, so the protocol is **enforced at run time**: a backtester refuses, at construction, an object missing any member. Tests also check that a delegating wrapper that does not inherit `BaseModel` backtests exactly like the model it wraps, and an `ast` scan of the source checks that the backtest layer reads no model attribute outside the protocol.

6.2 Why not a protocol for every layer up front

Not adopted	Reason
A protocol per layer, the root class as its default implementation	Every other seam has one implementation. A protocol there would restate the root class's public methods: one more declaration to keep in step.
All protocols in one <code>protocols.py</code>	A protocol owned by no caller drifts towards a copy of the implementing class's public surface.
Protocols at the level of the template methods	Root classes are not just signatures; they carry invariants . The dataset and factor roots own reading, warm-up, resampling and the store; <code>PortfolioConstructor.decide</code> owns the no-look-ahead and locked-position checks (chapter 7). A protocol there would hand the invariants to every implementer.

6.3 Ensembles: composition, not inheritance

`BaseEnsemble` is a class of its own that composes models and satisfies `Predictor`. Its combination rule needs one idea: the ensemble's labels are the **union** of its members' labels, and each label is combined from the members that predict it. With several members on one label, each is standardised per bar and the results averaged; with one member, its prediction passes through unchanged.

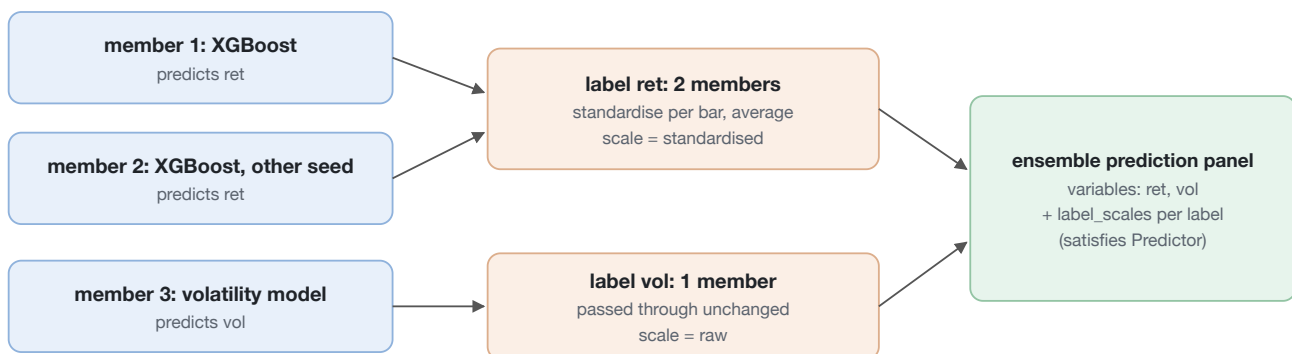


Figure 6-2. An ensemble combines per label: average when several members predict it, pass through when one does.

"Several seeds", "different models" and "one model for returns, another for volatility" are then **one rule**: no mode switch and no second composition class. The cost: a prediction's units depend on how many members predict it (averaged predictions are standardised scores, passed-through ones keep their units), so the protocol carries a per-label `label_scales`, and a rule that needs return units must check it.

7 Invariants enforced by interface shape: "only t is visible"

Chapter 0 called look-ahead a silent bug. This chapter shows how the project rules it out with **function signatures and data structures** rather than developer discipline.

7.1 A per-bar decision interface

There are two ways to design the interface that turns predictions into weights:

```
# A. Whole-panel interface: every date at once
weights = selector.select(scores_panel, next_fill_price_panel, rebalance_mask)

# B. Per-bar interface: one bar's context at a time
weights_t = rule.construct(context_at_t)
```

The signature of A lets an implementation read past `t`; an implementation of B **cannot even get hold of** future data. quantlab uses B, for two more reasons: an optimiser solves one problem per rebalance anyway, and a future event-driven backtester or live executor can call the same method unchanged.

`PortfolioContext` is a dataclass that describes one bar:

```
@dataclass
class PortfolioContext:
    timestamp: pd.Timestamp          # the bar decided on; the weights returned fill at the
    next_bar                         # each label's prediction at the bar, on symbol only
    predictions: xr.Dataset          # whether each stock can trade at the bar, judged from t
    tradable: xr.DataArray           # what is actually held
    and before                       # a bounded window of returns up to t
    current_weights: xr.DataArray    # bars since the last real price (grows during a halt)
    returns: xr.DataArray | None     # values at t of the factors the rule declared
    staleness: xr.DataArray | None
    factors: xr.Dataset | None
    risk_exposures: xr.Dataset | None
```

(Fields and comments abridged; the full definition is in `quantlab/portfolio/base.py`.)

7.2 `decide()`: a checking shell that is not overridden

A portfolio rule implements `construct`, but callers always call `decide`, a base-class method that subclasses **do not override**. It wraps `construct` and does two things:

- it turns a `PortfolioConstructionError` (an optimiser with no solution, say) into "hold the current position at this bar" and records why, instead of crashing the whole backtest;
- it checks the returned row of weights: the stock axis matches the context; NaN and finite values are not mixed; **a locked position is not changed** (a stock held but not tradable now, during a halt for example); no weight goes to a stock that is neither tradable nor held. Breaking any of these is a bug in the rule and raises `ValueError`.

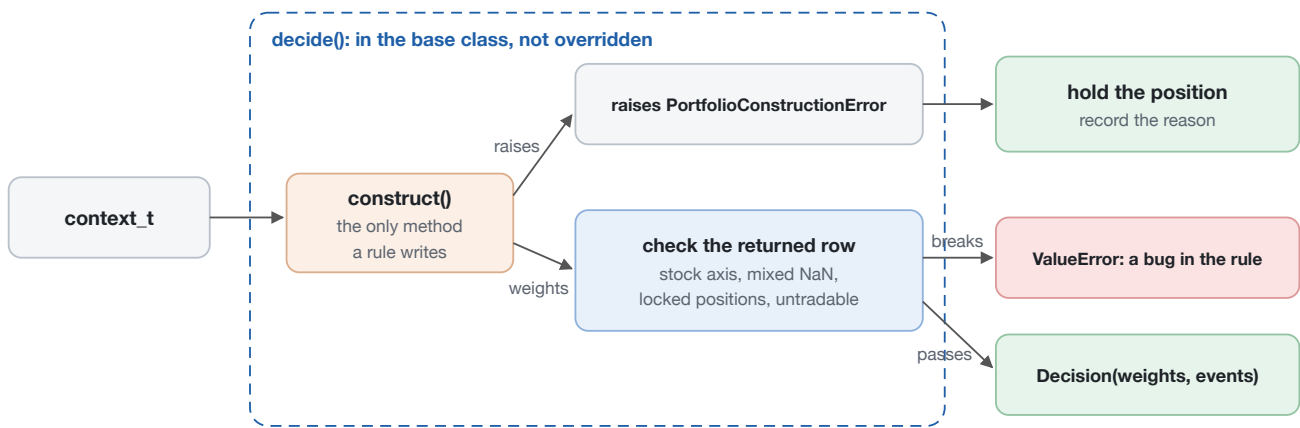


Figure 7-1. The flow of `decide()`. A rule's author writes only `construct`; failure handling and validity checks are done once, by the base class.

This is what chapter 6 meant by "root classes carry invariants": any new rule anyone writes is protected by these checks as long as it is called through `decide`.

7.3 Decision and execution are separate

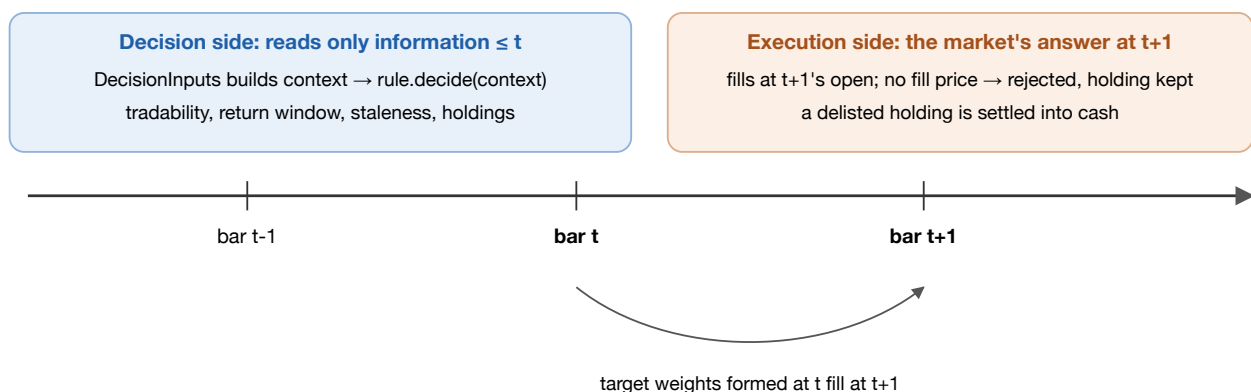


Figure 7-2. The decision side reads only t ; the execution side may read $t+1$. Both use one copy of the fill rules in `quantlab/execution`.

An example of a harmless-looking mistake: define "can this stock be picked at t " as "it has a fill price at $t+1$ ". Every target weight is then guaranteed to fill, the code is simple, and target weights equal holdings. But it tells the strategy at t which stocks will halt or delist at $t+1$, so it never buys the day before a halt. That is an optimistic bias, **and it cannot be computed in live trading at all**. `quantlab` instead:

- judges tradability on the decision side from information at t only;
- simulates faithfully on the execution side: an order without a fill price is rejected and expires, the holding is kept; a delisted holding is settled into cash at its last valuation;
- since a rule must know what is actually held, an `ExecutionBook` replays fills, rejections, delisting settlements and the cash limit with **exactly the rules the backtest engine uses**. Those rules live in one place, `quantlab/execution/rules.py`, down to `vectorbt`'s numeric details (floating-point comparison, how amounts are added).

7.4 DecisionInputs: one assembly point for research and live trading

What a decision may read is assembled by one module, `DecisionInputs` in `quantlab/portfolio/decision_inputs.py`, which serves both research backtests and live executors. The live executor is a separate repository by the same author, `quantlab-ibkr`, which connects quantlab's strategies to the broker Interactive Brokers.

Method	Use
<code>weights(predictions)</code>	Research backtest: decide every bar of a whole panel
<code>context(t, predictions, current_weights)</code>	Live executor: build one bar's context; the executor supplies the holdings of its own account
<code>rebalances(t)</code>	The schedule: is t a rebalance bar
<code>from_run(run_dir)</code>	Rebuild a recorded backtest's decision inputs from its run directory, without loading the model

It is the only code that builds a `PortfolioContext`. Each bar reads only the last `historyBars` raw prices (a number the rule declares), forward-filled within that window only. So **a decision depends only on what is known at t, never on where the caller's history starts**. A live executor has no natural start date, which makes this essential.

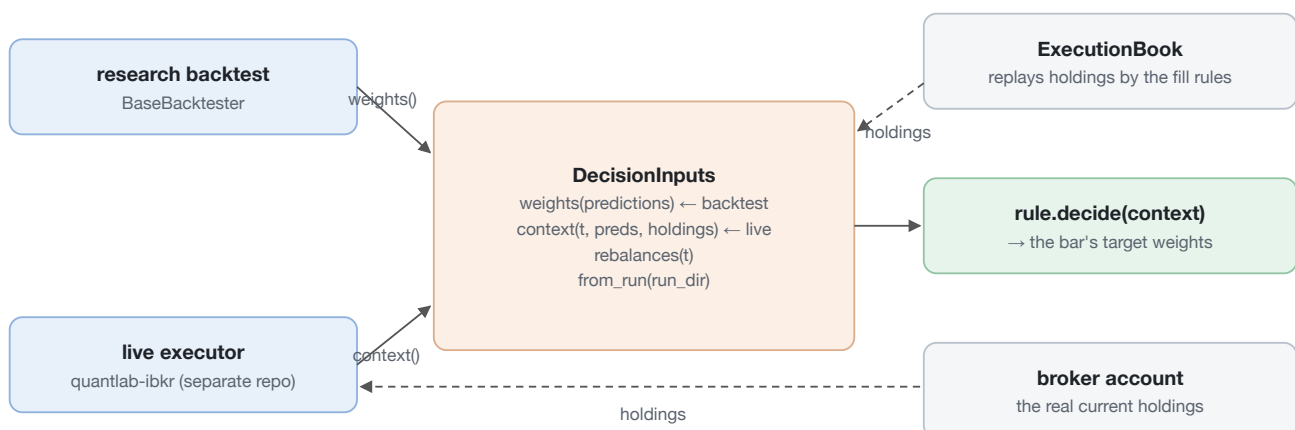


Figure 7-3. `DecisionInputs` serves both the research backtest and the live executor; they differ only in where the current holdings come from.

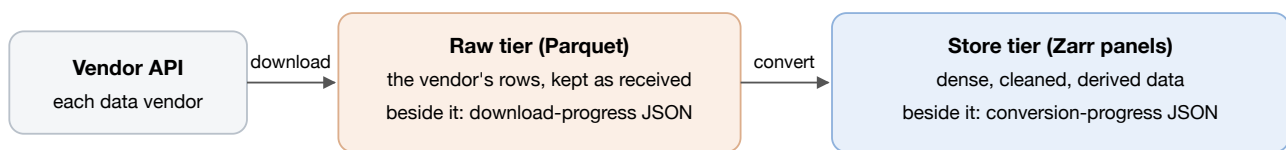
The same idea in the risk model

A factor risk model is a standard industry way to estimate risk (quantlab implements MSCI Barra's USE4 method): it explains how stocks move together through a set of common risk factors. Here it is a "data product with one row per bar": computed ahead, stored, each row using nothing later than its bar and only a bounded window. Computing a whole history at once and computing part of it then appending a row a day give identical rows, so a live executor appending one row a day stays aligned with research. The forecast at a bar, $B F B^T + \text{diag}(D)$, is one value type of the risk layer, `FactorRiskForecast`, used by portfolio optimisation, risk attribution and model tests alike; the rule for "which stocks this forecast covers" is written in that one place.

8 Robust long-running jobs: resumable work and atomic writes

Downloading a whole market takes thousands of requests and can run for hours; converting raw data into Zarr is a long job too. The requirement: **kill the process at any moment and a re-run carries on, never re-fetching finished work and never skipping unfinished work.**

8.1 Two storage tiers



A change of cleaning or filtering is a re-conversion, not a re-download

Figure 8-1. The raw tier and the store tier are separate.

8.2 Three progress ledgers

Ledger	What it records	The property it guarantees
Watermark	The date range of each stock already on disk	A watermark is written only after a whole batch is fetched and written. A stock interrupted halfway has no watermark and is fetched again.
Page ledger	Progress of one multi-stock request that the vendor pages: pages fetched, the next page's token, the files each page was written to	See the list below.
Chunk ledger	The time windows a Zarr conversion has appended	The symbol axis is fixed before the first window so all windows line up by column; when ledger and store disagree, resuming is refused, trusting neither alone.

Each design point of the page ledger guards against one specific failure:

- **Deterministic file names:** a shard is named `part-{batch_key}-{page:05d}.pqt`, where `batch_key` hashes the vendor, frequency, dates and the sorted stock list, with no timestamp or random part. Re-fetching a page overwrites the same file, so a retry can waste requests but never duplicates rows.
- **Data first, ledger second:** a crash between the two costs one re-fetched page; the opposite order could record a page that never reached disk.
- **The ledger remembers a fingerprint of its stock list:** opened with another list, it reads back empty instead of resuming someone else's progress.
- **Check before resuming:** every shard file the ledger records must still exist, or it raises. Skipping it would leave a hole no later read could detect.

- **No infinite loops:** a vendor that returns the token it was given raises `ValueError` instead of looping and writing a new shard every round.
- **Graceful degradation:** a missing or corrupt ledger file reads as an empty ledger, which costs that one batch a re-fetch.

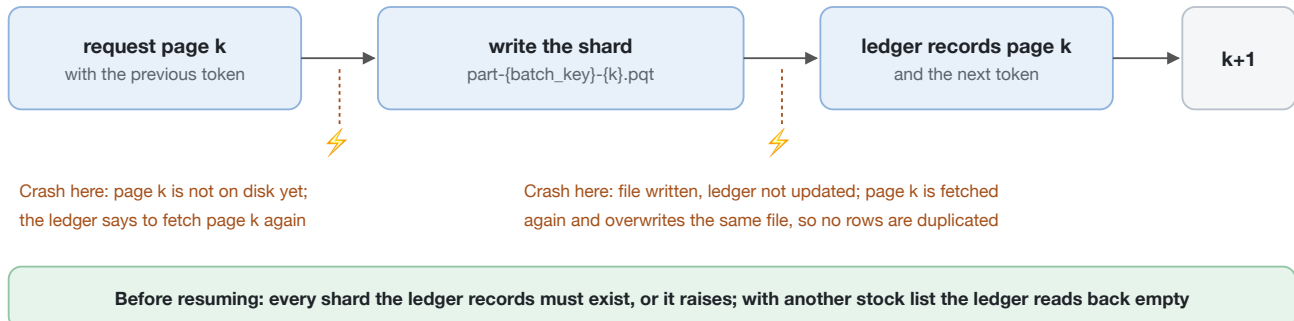


Figure 8-2. The page ledger writes data first and the ledger second. A crash at any step costs one re-fetched page.

8.3 Atomic writes, and "build the new before deleting the old"

- Every JSON sidecar is written by `write_json_atomically`: to a temporary file in the **same directory**, then `flush + fsync`, then `os.replace` over the target. The same directory, because a rename is atomic only within one file system. After a crash there is only ever the complete old file or the complete new one.
- Directories likewise: a backtest's result directory is written as a hidden `.{name}.partial` and renamed into place once every artifact has succeeded; any exception, `KeyboardInterrupt` included, removes the staging directory. **A result directory is complete or absent.**
- When a store is rebuilt, **the old store is deleted only after the new one is complete**: adding stock columns writes a `.widening.tmp` beside it and swaps by two renames; a full rebuild renames the old store to `.superseded.tmp` and renames it back on failure. If even that fails, it logs instead of raising, so as not to hide the real error, and the log names both copies.
- The rebuild tool's `data_root` is required, with no default: run from a code checkout with no data directory, it must never report success against a tree it never read.

One JSON file



A whole run directory

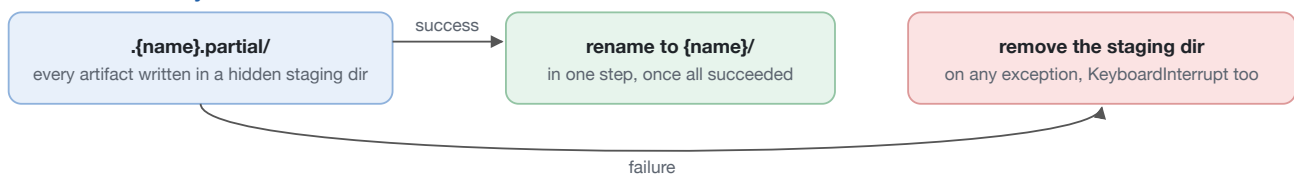


Figure 8-3. Atomic writes in two forms: a file through a temporary file and `os.replace`, a directory through a staging directory and rename. The result is complete or absent.

8.4 Measure first, then design: computing on the server

Tick quote data is huge: ten years of NBBO (the US national best bid and offer) for the S&P 500 is about 9 TB of CSV, while the training server reaches WRDS at only 5–6 MB/s in total, so downloading ticks and resampling locally would take more than three weeks. Measured on one day of 96 stocks: aggregated into one-minute bars by SQL on the WRDS server, 1.7 MB returns in 63 s; the raw ticks are 328 MB in 110 s.

So there is a separate data type, `nbbo_bars` : SQL on the server does the aggregation, reproducing all 13 variables of the local resampler, and the resulting store has the same layout as the local path's. The local tick path is kept as the reference the SQL is checked against, and the two agree bar for bar.

9 The run layer: a reproducible on-disk format

Whether a result can be reproduced depends on three things: **config, data and code**. Recording all three is the job of one layer, `quantlab/runs`. A training or a backtest is a "run", and everything it produces goes into one "run directory".

9.1 Two kinds of run, one mechanism

Trained run (TrainedRun)

```
XGBoostRegressor_trial_<time>/
  config.json rebuild recipe
  model.joblib checkpoint
  run.json written last

walk-forward train_cv:
  ..._trial_<time>/
    fold_0/ fold_1/ ...
    run.json kind=walk_forward

ensemble:
  member_0/ member_1/ ...
  run.json kind=ensemble
```

Backtest run (BacktestRun)

```
<BacktesterClass>_<time>/
  config.json rebuild recipe only
  weights.zarr target weights
  holdings.zarr actual holdings
  equity.zarr portfolio value
  predictions.zarr what the rule read
  metrics.json settlements.json
  report.html
  inputs/ copies of in-memory data
  run.json written last

run.json: kind, window, data
fingerprint, code record, trained run
```

Figure 9-1. The two kinds of run directory. The `run.json` header (format version, kind, time written) is shared, and `run.json` is written last as the "complete" marker.

`quantlab/runs/directory.py` is the mechanism both kinds share: one `run.json` header and format version; `run.json` written last as the completion marker; writing into a staging directory and renaming it; paths inside the directory always relative (a run copied from the server to a laptop opens as it is); an unknown format version refused with a message to re-run; and `open_run(path)`, which dispatches on `kind` to the right reader through a fixed kind-to-class table, importing the class only when it opens a run.

9.2 File ownership: only one module knows the file names

The rule: **files of a run directory are read and written only through `quantlab/runs`, and no other code may contain their names**. With file names and JSON keys scattered over writers and readers, two definitions drift apart, and callers guess the layout with things like `Path(checkpoint).parent.parent`, which breaks as soon as the nesting changes.

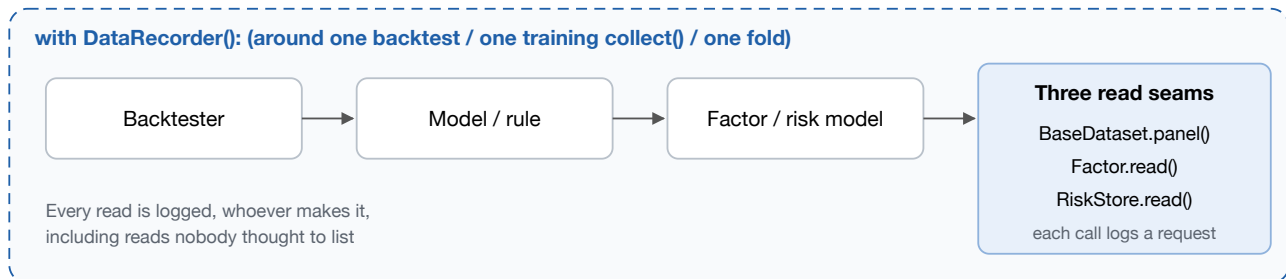
A test enforces it: it scans the library, the examples and the Python code in the docs (docstrings included), and fails on a string literal naming a file such as `run.json`, or on walking two levels up from a checkpoint with `.parent.parent`. Callers use typed accessors only:

```
run = BacktestRun.open(run_dir)
run.window, run.data_fingerprint, run.trained_run() # typed properties, never JSON indexed
by string keys
```

9.3 Data fingerprints: recorded where data is read

One way to learn what data a run used is to have the backtester **enumerate** every input to record. Nobody can enumerate them all: tradability data, the price history a rule reads before the window, each input of a merged dataset, every member of an ensemble... and the backtester would have to know factor internals.

quantlab does the opposite: **no enumeration; record data where it is read**.



On exit: each distinct request is read again, hashed with SHA-256 and written to `run.json`, keyed by component path

On rebuild: recomputed and compared; a difference only warns (revised data must still be backtestable)

Figure 9-2. Data fingerprints are recorded at the read seams; a new model or portfolio rule does nothing to be fingerprinted.

Engineering details of the hashing

- **Two-level digests:** one SHA-256 per variable; the request's digest hashes the time axis, the stock axis and each variable's name, dtype and digest. A warning can therefore name the variables that changed.
- **Canonical form:** values are hashed in their stored dtype (no up-casting); every NaN becomes one bit pattern and `-0.0` becomes `0.0`, or two reads of identical data could hash differently.
- **Bounded memory, parallel:** each variable is read and hashed in blocks of about 64 MB of whole bars (aligned to the store's chunks), and variables are hashed in parallel threads. Neither the blocking nor the parallelism changes a digest.
- **Leaves only:** only real stores or in-memory data are recorded; a merged dataset records nothing itself, its inputs do. Bookkeeping reads such as calendar lookups go through a separate `calendar()` and are not recorded; reads to exclude are wrapped in `unrecorded()`.
- **Merging per-bar reads:** a risk model's store is read one row per bar. Reads that differ only in range are merged into one request covering the first to the last bar, recorded and hashed once.

9.4 Code records

Every `run.json` also has a `code` part:

- `git`: the quantlab commit and whether tracked files had uncommitted changes (context only, never compared);
- `modules`: the SHA-256 of the source file of every class in the component tree and of its base classes, marked as a framework or a component module, with the component paths using it;
- `libraries`: the versions of numpy, pandas, xarray, polars, xgboost, torch, vectorbt, KunQuant and cvxpy.

A rebuild compares them item by item and warns on each difference, listing component modules before framework modules. Data and code follow one policy: **warn, never block**. Revised data must still be

backtestable, and a check that blocks only teaches people to work around it.

9.5 The full loop

```
run = BacktestRun.open(run_dir)
again = run.rebuild_backtester()          # rebuild the whole component tree from config.json
again.run()                              # recompute fingerprints and code record, compare,
warn on differences

inputs = DecisionInputs.from_run(run_dir) # live executor: rebuild the decision inputs without
the model
```

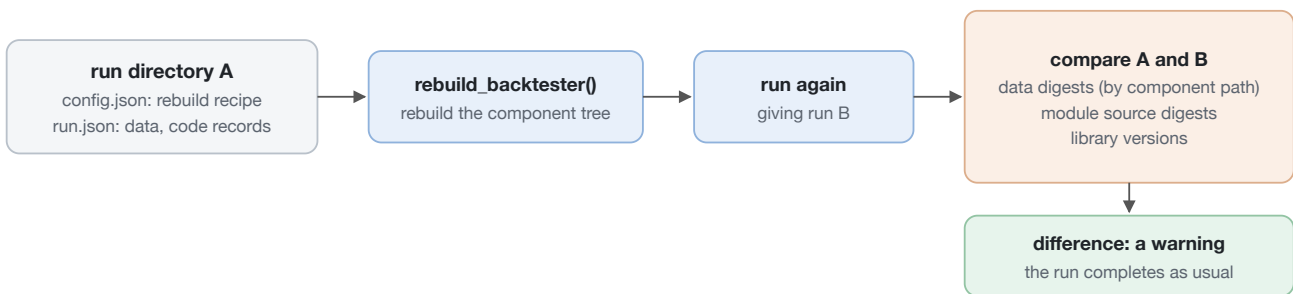


Figure 9-3. The reproduction loop: rebuild from a run directory, run again, compare the records; a difference only warns.

10 Isolating external dependencies

The project depends on several heavy or sensitive libraries and services. The principle: **each external dependency is imported by exactly one module, and a test enforces it.**

Dependency	Its one importing module	Reason
vectorbt	<code>backtest/engine_vectorbt.py</code>	The engine is replaceable (an event-driven engine is planned); the numeric rules holdings replay needs live separately in <code>execution/rules.py</code> .
cvxpy	<code>portfolio/predefined/mean_variance.py</code>	Only the mean-variance optimiser needs a solver; other rules and the root classes never load it.
wandb / mlflow	<code>tracking/wandb.py</code> / <code>tracking/mlflow.py</code>	See 10.1.
torch	the torch part of the model layer	No root class or config of any layer loads torch; a subprocess test checks it.
Vendor clients	<code>acquisition/<vendor></code>	See 10.2.

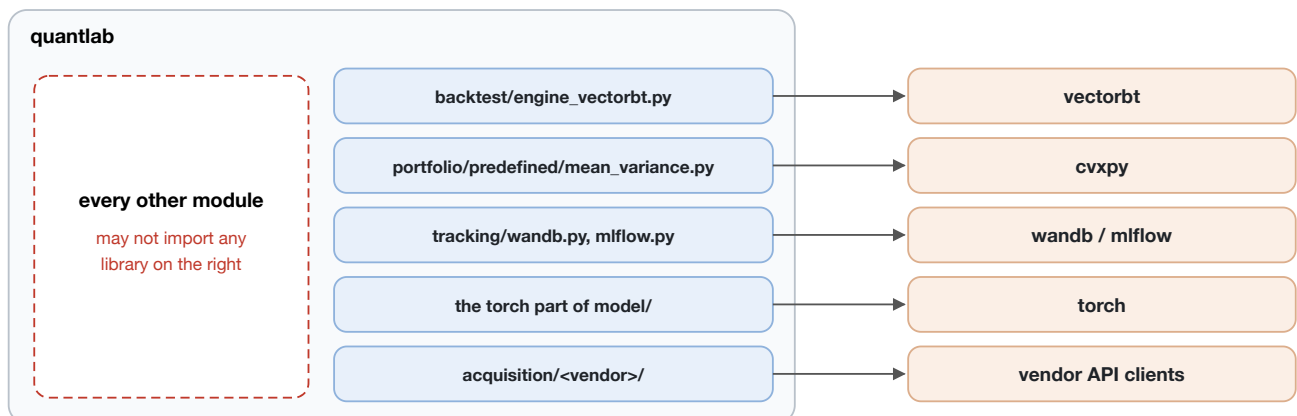


Figure 10-1. Each external dependency has one entry module. No other module may import these libraries, and a test enforces it.

10.1 Experiment tracking: chosen by config, off by default

- The tracker is a **component** in the config (the `tracker` field, serialised by class path like any other component); the default is a null tracker that does nothing. Unless a config names one, **no data leaves the machine**;
- W&B and MLflow are two adapters behind one small interface: `start_run` (a context manager), `log`, `summarize`, `update_config`, `log_table`, `log_file`;
- Not adopted, and why: an environment-variable switch (not reproducible from `config.json`); passing a tracker object to constructors without serialising it (a rebuilt run would silently stop tracking); a contextvar holding a "current run" (hidden global state, hard to reason about with deep-copied models).

10.2 Credentials and the credential-free inspector

- Every API key is read from environment variables only: never passed on the command line, never written to a config file, never logged.
- `SourceInspector` answers "what can this project download", and must be importable **without loading any vendor client**. It is checked by an AST scan of its own source plus a run-time look at `vars()`. Neither can see an import pulled in by a package `__init__.py`, so the three `__init__.py` files on its import chain must stay empty, and a test asserts it.

10.3 Platform differences live in docs and tests

On macOS, xgboost's wheel links Homebrew's OpenMP library (libomp) while torch bundles its own, and a process using both crashes or deadlocks. Only `OMP_NUM_THREADS=1` was measured to be safe (preloading libomp crashes torch instead). `tests/conftest.py` sets it on macOS before any import, and a test checks that the setting is there; Linux is unaffected.

11 Tests that guard the architecture

Besides ordinary functional tests, the suite has a set of **tests whose job is to guard architectural rules**. A rule written only in a document fades; a rule written as a test does not.

Test file (under tests/)	The rule it enforces
<code>test_layer_layout.py</code>	Every <code>quantlab</code> import (function-level ones included) follows the layer order; every module belongs to a layer; root classes load no torch; framework modules never import <code>predefined</code> ; <code>cvxpy</code> , <code>wandb</code> and <code>mlflow</code> each have one importer; <code>predefined/__init__.py</code> is empty.
<code>test_run_ownership.py</code>	Run-directory file names appear only in <code>quantlab/runs</code> ; no <code>.parent.parent</code> guessing.
<code>test_backtest_predictor_protocol.py</code>	A non-inheriting wrapper backtests identically; an object missing a member is refused; the backtest layer reads protocol members only.
<code>test_source_inspector.py</code>	The credential-free inspector reaches no vendor client; three <code>__init__.py</code> files are empty.
<code>test_data_recorder.py</code> and others	Recording and comparing data fingerprints; only the data, factor and fingerprint modules ask a factor what it reads.
<code>test_code_record.py</code>	The content and comparison of code records.
<code>test_undefined_names.py</code>	Undefined names and unused imports, checked with ruff.
<code>test_extensibility_contract.py</code> , <code>test_risk_contract.py</code> , <code>test_tracker_contract.py</code>	The contract of each extension point: what a user's new implementation must and need not provide.
Reference files (such as <code>topn_reference.npz</code>)	Numbers are bit-identical before and after a refactor; regenerated only when the computation's meaning changes on purpose.

What these tests have in common

- **Positive controls:** the layer test first asserts that `quantlab.model.base` is among the modules it checks, so a wrong path cannot make it pass forever on an empty scan.
- **Order, not paths:** the checks are about dependency order, so moving a file does not mean editing the test.
- **No exemption lists:** the rules do not allow a list of "these places may break it", because such lists are how rules erode.
- **Fully offline:** the whole suite needs no network and no credentials. Measurement tests that need real data are not collected by default; they run only when named on the command line and fail with a clear message when the data is missing.

Another guard rail: runnable documentation

Every public class and function has a numpypdoc docstring that **must include an example**, and the example output is real output, not written by hand. Python code in the docs is also subject to the file-ownership scan of 9.2.

12 Design records and project conventions

12.1 Architecture decision records: writing down "why"

`docs/adr/` holds 28 architecture decision records (ADRs). Each uses one format: what was decided, why, **which other options were considered and why they were not taken**, and the consequences. The "options not taken" part is the shortcut to understanding this repository: next to almost every design that looks odd at first sight is the reason the more obvious one does not work. Most of the "why" in this document comes from there.

Topic	Where in this document
Panels are read by date range, never by rewriting another object's config	Chapter 4
A label is a forward-shifted factor; splits purge by its lookahead	5.2
Torch models choose their sample shape; the base keeps the data contracts	5.3
The backtester depends on a Predictor protocol; when to define a protocol	Chapter 6
The api layer is a DataFrame-in, DataFrame-out facade	2.4
Per-bar decisions; decision and execution apart; one assembly of decision inputs	Chapter 7
The tracker is chosen by config and off by default	10.1
Run-file ownership, component declarations, data fingerprints and code records	Chapters 3 and 9
Layers form an ordered dependency chain	1.2
A factor risk model is a per-bar data product	7.4
NBBO bars are aggregated on the server	Chapter 8

12.2 One vocabulary

`CONTEXT.md` at the repository root is a glossary: it fixes which word names each concept and lists the synonyms to avoid. For example, a "panel" is xarray data shaped `(timestamp, symbol)` while a "frame" is the caller's DataFrame; a "weight" is the target fraction asked for while a "holding" is the fraction actually held after fills; "lookahead" and "span" mean different things. Code, docs and scripts all use one vocabulary; appendix B lists part of it.

12.3 No backward compatibility

The project is not in production yet, so its policy is explicit: **when a breaking refactor is needed, do it directly: no config migrations, no compatibility layers, no old names kept.** A config or run directory in an old format is refused with a message to re-run or retrain. The benefit is that the code holds no layers of forwarding and compatibility branches, and changing the architecture is cheap; the cost is that past runs must be recomputed before they open again.

12.4 Development process

Larger changes follow "question the design hard, then write a spec (a GitHub issue), then split it into small, separately committable tasks, then implement". Research experiments are recorded in a separate private repository, one directory per experiment with its setup, the scripts actually run and the conclusions, apart from the library code.

13 Trade-offs at a glance

No design is free. The table puts the main decisions of this document side by side: the middle column is what was gained, the right column what was paid.

Decision	Gained	Paid
Layers exchange only xarray panels	One representation of missing data; vectorised cross-sections; models consume it directly; Zarr loads on demand	DataFrame users need a facade; dense panels cost memory on sparse data (many years of the whole market, many delisted stocks)
Config dataclasses + component declarations	Every run serialisable and rebuildable; the component tree is walkable	Every configurable object needs a config class; moving a class invalidates saved configs
Reads are queries with arguments; configs frozen	No shared mutable state; one dataset serves many users; several inputs can merge	Every read reopens the store; callers pass their own dates
Labels are a <code>Forward</code> wrapper	Any factor can be a label; one source for the lookahead; automatic purge	Bars lost at every split; the wrapped factor must use only data up to <code>t</code> , which <code>Forward</code> cannot check
Per-bar decision interface	The future is structurally invisible; research and live trading share one method	No whole-panel vectorised shortcut; slower
Rejections and delistings simulated faithfully	No optimistic bias; computable live	Target weights $\neq$ holdings: holdings must be replayed and recorded separately
A protocol only when a second implementation appears	Small interfaces; invariants stay in root classes	No static type checking; protocols rely on run-time checks and tests
Run directories as a layer	File names defined once; one format; directories movable	A new kind of run artifact must go through the runs layer
Fingerprints recorded at the read seams	Nothing missed; zero cost for new components	Each request is read again and hashed at the end: extra I/O
Differences warn, never block	Revised data can still be backtested	Warnings can be ignored
Training data on GPU / in RAM	Reads one to two orders of magnitude faster (measured)	Data larger than RAM needs another design
No backward compatibility	Fast refactors; no compatibility layers in code	Old runs must be recomputed to open
Architecture rules written as tests	Rules are neither forgotten nor quietly broken	More test code than library code; changing the architecture means changing the guard tests too

14 Strengths, limitations and reserved directions

14.1 Strengths

1. **Correctness from structure, not discipline.** No look-ahead, no duplicated data, no half-written files: interface shapes, single assembly points, atomic operations and guard tests ensure them. A newcomer who does not know every pitfall will still find these bugs hard to write.
2. **Reproducibility is first-class.** Config, data and code are all recorded; a run directory rebuilds into live objects and differences are reported automatically.
3. **Extension is cheap and even-handed.** A new factor, model or portfolio rule mostly means one to three hooks; shipped and user implementations take the same path, with no internal privileges.
4. **Research and live trading share code.** A KunQuant factor's batch and streaming modes come from one graph; the per-bar rule interface, `DecisionInputs` and the fill rules are reused directly by the live executor.
5. **Clear dependency boundaries.** Layers form an acyclic chain, each heavy library has one entry point, and opening a run directory loads no torch.
6. **Decisions are traceable.** The decision records give the alternatives and measurements behind each key choice, so a reader of the code can learn why it is not written another way.

14.2 Limitations

- **Heavy dependencies, steep start.** KunQuant needs a C++ compiler at run time and its factor tests compile for minutes; Python ≥ 3.13 is required; the main data sources need paid accounts. The repository does include a Yahoo Finance example that needs no account, and an offline quickstart on synthetic data.
- **Single process, single machine.** No service layer, no job queue; training and factor computation assume one large-memory workstation with a GPU.
- **No static type checking.** Protocols and contracts rely on run-time checks and tests; adopting pyright or mypy is treated as a larger, separate decision.
- **Interfaces still change.** No backward compatibility means outside users must follow the main branch.
- **The backtest is vectorised.** Partial fills and intraday matching need an event-driven engine; the current engine lets a rejected order expire instead of retrying it on later bars.

14.3 Directions the architecture reserves

- **Event-driven backtesting** (NautilusTrader is planned): the per-bar rule interface and `DecisionInputs.context` are ready for it.
- **A service and multiple users:** stateless reads, frozen configs and serialisable components are the preconditions for editing and testing factors and models online.
- **Unstructured data** (news, say): a dataset's cleaning steps can be overridden; nothing assumes market data is always open-high-low-close-volume.
- **A web front end:** explicitly out of scope for now.

App. Appendix

A. Suggested reading path

1. `README.md` and `docs/user-guide/concepts.md` : the overall concepts.
2. `CONTEXT.md` : the glossary, to agree on words first.
3. `tests/test_layer_layout.py` : every layer and its order at a glance.
4. `quantlab/core/component.py` : the component rule, serialisation and rebuilding.
5. `quantlab/dataset/base.py` → `quantlab/factor/base.py` → `quantlab/label/forward.py` : how data is read.
6. `quantlab/model/base.py`, then `torch_model.py` / `library_model.py` : template methods and hooks.
7. `quantlab/portfolio/base.py` and `decision_inputs.py` : the no-look-ahead interface design.
8. `quantlab/backtest/base.py` : the Predictor protocol and the backtest workflow.
9. `quantlab/runs/directory.py` and `record.py` : the reproducibility machinery.
10. `docs/adr/` : whenever you ask "why is it written this way".
11. `docs/developer-guide/extending.md` : writing your own data source, factor, model or portfolio rule.

B. Glossary

Term	Meaning
panel	An <code>xarray.Dataset</code> with exactly the dimensions (timestamp, symbol)
frame	The caller's long pandas / polars table; appears only in the api layer
store	A Zarr directory on disk holding one panel
component	An object built from one config dataclass, with <code>get_config</code> / <code>rebuild</code>
warm-up	Bars read before the first requested bar so rolling computations have enough history; counted in bars
lookahead / span / delay	How many bars after t a label at t reads (= delay + span) / how many bars it accumulates over / the bars between the signal and the first bar counted
purge	Dropping the last lookahead bars of the earlier segment at every split
Predictor	The protocol of a signal source a backtest can use; models and ensembles satisfy it
weight / holding	The target fraction asked for / the fraction actually held after fills
locked position	A stock held but not tradable at the bar; a rule must keep its weight
decision inputs	Everything a rule may read at t except holdings; assembled only by <code>DecisionInputs</code>
trained run / backtest run	The two kinds of run directory, each described by its <code>run.json</code>
data fingerprint / code record	Digests of the data a run read / digests of the code it used, plus library versions
tracker	W&B, MLflow, or nothing (the default)

C. A minimal end-to-end example

From the repository README (`examples/yahoo_us_equity.py` ; no account needed, about 20 seconds).
Every step builds a config object and calls a public method:

```
# 1. Data: a DataFrame becomes an in-memory MarketDataset
prices = FrameDataset(stocks_df, columns=COLUMNS)

# 2. Factor and label, both built from a FactorConfig; Return is a ready-made return label
factor = Alpha158Stock(FactorConfig(dataset=prices, warmup_bars=60, mode="batch",
                                     data_columns=ADJUSTED, file_path="../../../alpha158.zarr"))
label = Return(FactorConfig(dataset=prices, warmup_bars=0, mode="batch",
                             data_columns=("adjOpen",), kwargs={"n_forward_periods": 5},
                             file_path="../../../ret_5.zarr"))

# 3. Model: dates are set on the outermost object only and passed down with each read
model = XGBoostRegressor(ModelConfig(
    factors=[factor], labels=[label], model_save_dir="../../../models",
    hyperparameters={"num_boost_round": 200, "max_depth": 3, "eta": 0.03},
    train_start="2016-01-01", train_end="2021-12-31",
    test_start="2022-01-01", test_end="2022-12-31", ...))
model.collect()
checkpoint = model.train()          # writes a trained run directory

# 4. Portfolio rule and backtest: the rule is a component in the config
backtester = USEquityCrossectionSelectStockVectorBt(CrossSectionBacktestConfig(
    price_dataset=prices, model=..., model_mode="load", checkpoint=str(checkpoint),
    start_date="2023-01-01", end_date="2024-12-31", output_dir="../../../backtests",
    rebalance_periods=5,
    constructor=TopNConstructor(TopNConfig(direction="long_only", top_n=10)),
))
result = backtester.run()          # writes a backtest run directory

# 5. Reproduce: rebuild from the run directory and run again, same equity curve
BacktestRun.open(run_dir).rebuild_backtester().run()
```

Compiled from the repository's source code, architecture decision records (`docs/adr/`), glossary (`CONTEXT.md`), developer documentation and tests. Every measured figure is quoted from the repository's documentation; none was measured for this document. Illustrative code is marked as such; other snippets are taken from the repository.